📝 Node.js & Setup

Welcome to your first backend exercises! These tasks will help you practice what we covered in the Node.js lecture. Follow the steps carefully, test your work, and be proud of every line you write.

🟢 Part 1: Setup & Basics

☑ 1. Verify Installation

- Open your terminal.
- Run the following commands:

```
node -v
npm -v
```

- Make sure both versions appear (if not, ask your instructor).

☑ 2. Initialize Your Project

- Create a folder named `first-node-app`.
- Open it in VS Code.
- Run:

```
npm init -y
```

- In `package.json`, add this line:

```
"type": "module"
```

☑ 3. Create a Basic Server

- Inside the folder, create a file called `server.mjs`.
- Create a Node.js HTTP server that:
  - Responds with "Welcome Home" when visiting `/`
  - Responds with "About You" when visiting `/about`
  - Responds with "Page not found" for any other path

- Use `res.writeHead()` and `res.end()` to send responses.

🟢 Part 2: Intermediate Tasks

☑ 4. Add Request Logging

- Inside your server, add a `console.log()` to print the method and URL of every request. Example:

```
Request received: GET /
```

☑ 5. Add a JSON API Route

- Add a route `/api` to your server.
- When visited, it should return a JSON object like:

```
{
  "course": "Full Stack",
  "technology": "Node.js",
  "instructor": "Dania"
}
```

🟢 Part 3: Advanced Tasks

☑ 6. Serve an HTML File

- Create a file called `index.html` with a short greeting and your name.
- Modify your server to serve this file when visiting `/`.
- Use `fs.readFile()` to read and send the file.

☑ 7. Handle File Errors Gracefully

- If the HTML file is missing, your server should:
  - Return a status code `500`
  - Send the message: `Server is on fire 🔥!`

🌟 Bonus Challenge (Optional)

☑ 8. Create a Dynamic Greeting Route

- Add a route like `/greet?name=Dania`
- Your server should respond with: `"Hello, Dania!"`

- Tip: Use the `URL` object to extract the query parameter.